

Основные определения

ООП - парадигма программирования, основанная на концепции объектов, которые взаимодействуют друг с другом в ходе выполнения программы.

Объект - то, что существует в предметной области и может быть идентифицировано по атрибутам или поведению.

Атрибуты - данные объекта, описывают его свойства, состояние.

Методы - определяют поведение объекта, что он может делать.

Класс - модель для создания объектов, описывающая их структуры и определяющая алгоритмы для работы с этими объектами.

Реализация классов в C++

Класс в C++ - создаваемый программистом тип данных, который включает данные (атрибуты) и функции (методы).

Объект в C++ - переменная типа класса.

```
class ИмяКласса {  
    // спецификаторы области доступа  
    private: // default  
    protected:  
    public:  
  
    // создание атрибутов и методов  
};
```

```
class Poind2d {
    // приватный атрибут
    double x{0}, y{0};

    public:
        // метод
        // const указывает, что состояние объекта не
        поменяется
        double dist_to_00() const {
            return sqrt(pow(x, 2) + pow(y, 2));
        }
};

int main() {
    Poind2d point;

    cout << point.dist_to_00(); // 0
    return 0;
}
```

Базовые принципы

Абстракция - описывает классы объектов, выделяем особенности, важные для задачи.

Инкапсуляция - механизм, который объединяет данные и код, работающий с этими данными, а также защищает их от внешнего вмешательства или неправильного использования.

Наследование - возможность создать новый класс на основе существующего, при этом свойства и функциональность родительского класса заимствуются новым классом.

Полиморфизм - возможность для объектов с одинаковой спецификацией иметь различную реализацию. (одно и то же имя можно использовать для решения схожих, но технически разных задач).

1. Абстракция

Машина -> Легковая машина -> конкретная машина (объект)

2. Инкапсуляция

Что дает?

- Данные и функции, работающие с этими данными лежат вместе
- Обеспечивает сокрытие внутренней структуры данных и защиту от произвольного внешнего воздействия на эти данные

Т.е. разработчик дает интерфейс (открытая часть) взаимодействия, но ограничивает доступ к реализации (закрытая часть)

Спецификаторы доступа

- **private** - элемент доступен только внутри класса
- **public** - элемент доступен везде
- **protected** - элемент доступен только внутри класса и его подклассов

Обеспечение доступа

- геттеры (getters) - обеспечивают чтение
- сеттеры (setters) - обеспечивают запись

```

class Poind2d {
    // приватный атрибут
    double x{0}, y{0};

public:
    double dist_to_00() const {
        return sqrt(pow(x, 2) + pow(y, 2));
    }

    double get_x() const {
        return x; // this->x - то же самое,
ТОЛЬКО ЯВНО
    }

    void set_x(double x) {
        this->x = x; // берем x объекта и
обновляем
    }

    // для y тоже самое, только буква меняется
};

int main() {
    Poind2d point;

    point.set_x(3);
    point.set_y(4);

    cout << point.get_x() // 3
    cout << point.get_y() // 4

    cout << point.dist_to_00(); // 5
    return 0;
}

```

В геттеры и сеттеры можно встраивать дополнительную логику обработки. При прямом доступе так сделать не получится

Конструкторы

Конструктор - механизм, который позволяет инициализировать объект и разместить его в памяти.

Объявление, определение и особенности

1. имя конструктора = имя класса
2. нет типа результата (даже `void` 'а)
3. нет `return`
4. конструкторов может быть много

```
class Point2d {
    public:
        // конструктор по умолчанию
        Point2d() = default;

        Point2d(double x, double y);
        Point2d(string point_name, double x, double
y);

        // конструктор копирования
        // const + передача по ссылке – гарантия,
        // что ничего не поменяем и сэкономим память
        Point2d(const Point2d& point);

        ...
};

// определяем конструктор
Point2d::Point2d(string point_name, double x, double
```

```

y) {
    this->point_name = point_name;
    this->x = x;
    this->y = y;
}

// предыдущий способ допустим, но у нас же была
причина,
// по которой мы написали сеттеры...
Point2d::Point2d(string point_name, double x, double
y) {
    set_point_name(point_name);
    set_x(x);
    set_y(y);
}

// а еще можно так!
Point2d::Point2d(string point_name, double x, double
y)
    : Point2d{"", x, y} {
}

int main() {
    // без дефолтного конструктора работать не будет
    Point2d p1;

    // через функциональный вызов
    Point2d p2(3, 4);

    // через списов инициализации - считается best
practice
    Point2d p3{"A", 3, 4};

    // конструктор копирования
    Point2d p4(p1);

```

```
    return 0;  
}
```

Размещение в куче

```
Point2d* point_p = &p1;  
Point2d* point_p = new Point2d("Q", -3, -4);
```

Деструктор

Деструктор - механизм, который выполняет завершающее действие, освобождает память занятую объектом.

Объявление, определение и особенности

1. имя деструктора = ~имя класса
2. нет типа результата (даже `void` 'a')
3. нет `return`

```
class Poind2d {  
    public:  
        ~Poind2d() {  
            cout << "Объект удален!!1!1";  
        }  
        ...  
};
```

Friend-функции

Дружественные функции - функции, которые не являются членами класса, однако имеет доступ к его закрытым членам (все, что в `private`).

```
class Point2d {
    private:
        double x, y;

    public:
        // только объявляем
        friend double dist(
            const Point2d& p1,
            const Point2d& p2
        );
};

// определяем снаружи, не указываем Point2d:: как для
// методов и конструкторов
double dist(const Point2d& p1, const Point2d& p2) {
    return ...
}
```